DEEP LEARNING

Complete Exam Notes

(Hinglish Edition — Exam ke liye sab kuch)

IITM BS Degree | Week 1–8 | Quiz 2

CNN Calculations — SABSE ZYADA MARKS (30%)
Optimizer Numericals (AdaGrad, Adam, RMSProp) — 20%
Backpropagation — 15%
Regularization + Bias-Variance — 15%
Activation Functions — 10%
CNN Architectures + Convolution — 10%

CHAPTER 0: Deep Learning kya hai? — Zero se Shuru

Ye chapter un logon ke liye hai jinhone kabhi deep learning nahi padhi. Ek baar ye samjh lo — baaki sab formulas automatically connect ho jayenge.

Sabse Pehle: Machine Learning vs Deep Learning

Pehle normal programming samjho:

Normal Programming vs Machine Learning:

NORMAL PROGRAMMING:
  Tum rules likhte ho → Computer output deta hai
  Example: 'Agar temperature > 30 to AC on karo'

MACHINE LEARNING:
  Tum data dete ho → Computer khud rules seekhta hai
  Example: Hazaron ghar ka data do → Computer seekhta hai price predict karna

DEEP LEARNING:
  Machine Learning ka ek powerful type
  Brain ki tarah behave karta hai — bahut saare neurons connected
  Images, audio, text sab handle kar sakta hai

Brain se Inspiration — Biological Neuron

Deep Learning ka idea human brain se aaya hai. Brain mein ~86 billion neurons hain jo ek doosre se connected hain.

Brain ka Part	Neural Network ka Equivalent
Dendrite	Input connections — data receive karta hai
Soma (cell body)	Computation — numbers add karta hai, threshold check karta hai
Axon	Output — result agla neuron ko bhejta hai
Synapse	Weight — connection kitna strong hai
Neuron fire karna	Activation — output produce karna

Simple analogy: Sochlo tumhare paas ek dost hai jo bahut saare inputs sun ta hai (gossip, news, rumours) aur fir decide karta hai kya karna hai. Woh inputs sun ta hai (dendrites), process karta hai (soma), aur action leta hai (axon). Artificial neuron bilkul aisa hi kaam karta hai.

Ek Artificial Neuron — Step by Step

Ek single neuron ye 3 cheezein karta hai:

```
Step 1: COLLECT — sab inputs x1, x2, x3... lo

Step 2: WEIGHT & SUM — har input ko weight se multiply karo, phir add
karo

         z = w1*x1 + w2*x2 + w3*x3 + ... + bias

Step 3: ACTIVATE — result ko activation function se pass karo

         output = f(z)    [f = sigmoid, ReLU, tanh, etc.]
```

Concrete Example: Kya mujhe chai peeni chahiye?

Inputs: Weights (importance):

x1 = Thakawat hai? (1/0) w1 = 0.8 (thakawat important hai)

x2 = Sardi hai? (1/0) w2 = 0.6

x3 = Exam kal hai? (1/0) w3 = 0.9 (exam bahut important!)

z = 0.8×1 + 0.6×0 + 0.9×1 + bias = 1.7 + bias

Activation: agar z > threshold => output = 1 (chai lo!)

else => output = 0 (mat lo)

Weights kya hain aur 'Learning' ka matlab kya hai?

Weights = Numbers jo batate hain ki kaunsa input kitna important hai. Ye numbers shuru mein random hote hain.

Learning ka matlab — simply itna hi hai:

1. Network ko data dikho (e.g., hazaron cat aur dog photos)
2. Network galat jawab deta hai shuru mein (kyunki weights random hain)
3. Error measure karo — kitna galat tha?
4. Weights thoda adjust karo error kam karne ke liye
5. Yahi process lakhs baar repeat karo
6. Gradually weights sahi ho jaate hain — network 'seekh' jaata hai

Ye process = TRAINING

Weights adjust karne ka tarika = GRADIENT DESCENT

Error measure = LOSS FUNCTION

Network Structure — Layers ka Concept

Ek neuron se zyada powerful nahi banta. Isliye hum neurons ko layers mein organize karte hain:

Layer Type	Kaam kya hai
Input Layer (Layer 0)	Raw data andar aata hai. Ek neuron = ek feature. Image ke liye: ek neuron = ek pixel.
Hidden Layer(s)	Yahan asli magic hoti hai. Ye layers patterns seekhti hain — edges, shapes, faces, etc.
Output Layer	Final answer. Classification mein: ek neuron per class. Regression mein: ek neuron (value).

Deep Learning mein 'Deep' ka matlab: Multiple hidden layers hain. Zyada layers = zyada complex patterns seekh sakta hai.

Forward Pass aur Backward Pass — The Core Loop

Training mein do cheezein baar baar hoti hain:

FORWARD PASS	BACKWARD PASS
Input se Output tak jaana Har layer calculations karta hai Final output = prediction **Direction: Input → Output**	Output se Input ki taraf aana Error ka blame assign karna Weights update karna **Direction: Output → Input**
Example: Image dalo → 'Cat' output aaya	Example: Answer galat tha → weights adjust karo

Loss Function — Error Measure

Loss function batata hai ki network ka jawab kitna galat tha. Training ka goal = loss minimize karna.

Situation	Loss Function
Regression (number predict karna)	Mean Squared Error (MSE) — squared difference
Classification (category predict karna)	Cross-Entropy Loss — probability-based error

Simple analogy — Loss Function:

Socho tum darts khel rahe ho.

Loss = center se kitna door laga dart

Training = practice karte karte aim improve karna

Loss 0 = bullseye (perfect prediction!)

Loss high = target miss

Gradient Descent — Weights Kaise Update Hote Hain

Gradient = Loss ka slope. Ye batata hai ki loss kaunsi direction mein badh rahi hai.
Descent = Neeche aana (loss kam karna).

Pahad se neeche aane ki analogy:

Imagine karo tum ek pahad par andheron mein ho aur neeche jaana hai.

Tumhe sirf ye pata hai ki tumhare pair ke neeche slope kaunsi taraf hai.

Tum us direction mein ek chhota step lete ho jahan neeche ki taraf slope hai.

Baar baar ye karte ho jab tak valley mein nahi pahunch jaate.

Gradient Descent exactly yahi karta hai!

Gradient = slope ka direction

Learning rate (eta) = step ka size

Valley = minimum loss (best model)

Formula: weight_new = weight_old - eta × gradient

⚠ **EXAM TRAP:** Learning rate bahut BADA ho to: Steps bade honge, valley overshoot ho jayegi. | Learning rate bahut CHHOTA ho to: Bahut slow converge hoga, time waste. | Sahi learning rate = Goldilocks problem!

Kya Seekhta Hai Deep Learning?

Har hidden layer kuch alag seekhti hai — shallow se deep concepts ki taraf:

Layer	CNN mein kya seekhta hai (Image example)
Layer 1 (Shallow)	Simple edges aur lines detect karta hai
Layer 2	Corners, curves, simple shapes
Layer 3	Eyes, nose jaisi features
Layer 4 (Deep)	Poora face, object recognize karta hai

Yahi reason hai 'Deep' Learning: Depth ke saath abstract features seekhne ki ability badh ti hai.

Parameter kya hota hai?

Parameter = wo number jo training ke through seekha jaata hai. Network ke weights aur biases = parameters.

Parameter count ka scale samjho:

Tumhara IITM assignment model: ~thousands of parameters

AlexNet (2012): ~61 million parameters

GPT-3 (2020): ~175 billion parameters

GPT-4 (2023): ~Trillion+ parameters (estimated)

Zyada parameters = zyada complex patterns seekh sakta hai

Lekin = zyada data aur compute chahiye

Aur = overfitting ka risk bhi badh ta hai

Epoch, Batch, Iteration — Confusing Terms Clear Karo

Term	Matlab
Training Data	Wo examples jinse model seekhta hai (e.g., 60,000 images)
Batch / Mini-batch	Ek baar mein kitne examples process karo (e.g., 32 images)
Iteration / Step	Ek batch process karna + weights update karna = 1 step
Epoch	Poora training data ek baar dekh lena = 1 epoch
Learning Rate (eta)	Har step mein weight kitna change karo

Example: 60,000 images, batch size=100

1 epoch = 60,000 / 100 = 600 iterations

Agar 50 epochs train karo = 30,000 total weight updates

Vanilla GD: Poora data ek baar mein (slow for large datasets)

Stochastic GD: Ek ek sample (noisy, fast updates)

Mini-batch GD: Groups mein (best of both worlds!)

Overfitting vs Underfitting — Simplest Explanation

Ye concept exam mein repeatedly aata hai. Simple language mein samjho:

UNDERFITTING	GOOD FIT	OVERFITTING
Model zyada simple hai Training data bhi galat Test data bhi galat	Training data theek Test data bhi theek **Yahi chahiye!**	Training data perfect Test data bahut galat Model ratta maar raha hai!
High Bias **Low Variance**	**Low Bias** **Low Variance**	**Low Bias** **High Variance**

(Exam term!)	(Exam term!)	(Exam term!)

Real life analogy:

UNDERFITTING = Student ne kuch padha hi nahi. Exam mein fail.

GOOD FIT = Student ne concepts samjhe. Questions solve kar leta hai.

OVERFITTING = Student ne previous year questions ratta maare. Exact same questions aaye to full marks. Lekin naye twist aaye to fail.

Data: Train, Validation, Test — Kyun Teeno Chahiye?

Split	Use kab hota hai
Training Data (~70%)	Model in data pe seekhta hai — weights update hote hain
Validation Data (~15%)	Training ke beech check karo ki model generalize kar raha hai ya nahi. Hyperparameters tune karo.
Test Data (~15%)	Final evaluation. Isko training ke dauran KABHI mat dekho!

⚠ **EXAM TRAP:** Test data ko training/validation mein mat use karo — warna tum cheat kar rahe ho! Real exam mein agar test data dekha to model ka actual performance pta hi nahi chalega.

Key Deep Learning Vocabulary — Ek Jagah Sab

Word	Matlab
Neuron / Node	Ek single computation unit
Weight	Connection ki strength — training mein seekha jaata hai
Bias	Extra offset parameter — network ko flexible banata hai
Activation Function	Non-linearity add karta hai — ReLU, sigmoid, tanh
Layer	Neurons ka group
Forward Pass	Input se output tak data flow
Backward Pass	Error se weights tak gradient flow (backprop)
Loss / Cost	Prediction kitna galat hai ka measure
Gradient	Loss ka slope — kis direction mein badh raha hai
Learning Rate (eta)	Gradient descent mein step size
Epoch	Poora training data ek baar process karna
Batch	Ek baar mein process kiye gaye samples
Overfitting	Training data memorize, generalize nahi
Underfitting	Bahut simple model — training bhi galat
Regularization	Overfitting rokne ki technique

Hyperparameter	Training se pehle set karo: learning rate, batch size, etc.
Feature	Input data ka ek attribute (e.g., image ka pixel, text ka word)
Feature Map	CNN mein conv layer ka output
Kernel / Filter	CNN mein sliding window jo features detect karta hai
Pooling	Feature map ka size reduce karna
Softmax	Output probabilities mein convert karna (sum = 1)
One-Hot Encoding	Class label ko vector mein represent karna: [0,0,1,0,0]

Deep Learning Applications — Kahan Use Hota Hai?

Application	Deep Learning kaise help karta hai
Image Recognition	CNN — camera se objects detect karna (phone face unlock)
Speech Recognition	RNN/Transformer — boli hui baat text mein convert karna
Machine Translation	Transformer — Google Translate
Medical Diagnosis	CNN — X-ray, MRI mein disease detect karna
Recommendation System	Netflix, YouTube ki recommendations
Self Driving Cars	CNN + RL — road, signs, pedestrians detect karna
ChatGPT / Claude	Transformer — text understand aur generate karna
AlphaFold	CNN + Attention — protein structure predict karna

Is Course ka Road Map — Kya Kya Padha Hai?

Week	Topic
Week 1	History — MP Neuron, Perceptron, AI Winter aur Revival
Week 2	Multi-Layer Perceptrons — Hidden layers, XOR problem solve
Week 3	Sigmoid Neurons — Smooth activation, gradient descent
Week 4	Feed Forward Networks — Full structure, backpropagation
Week 5	Gradient Descent Types — Momentum, NAG, SGD variants
Week 6	Adaptive Learning Rates — AdaGrad, RMSProp, Adam
Week 7	Regularization — L2, Dropout, Early stopping
Week 8	CNN — Convolution, Pooling, Architectures (AlexNet, ResNet)

Exam mein sabse zyada kya aata hai (3 papers ka analysis):

1. CNN Calculations (Week 8) — 30% marks
2. Optimizer Numericals (Week 6) — 20% marks
3. Backpropagation (Week 4) — 15% marks
4. Regularization + Bias-Variance (Week 7) — 15% marks

5. Activation Functions (Week 3-6) — 10% marks
6. History aur Foundations (Week 1-2) — 10% marks

RNN, LSTM, Transformers (Weeks 9-12) = 0 marks in past 3 papers!

Ek Complete Mental Picture — Sab Kuch Connect Karo

```
Data (images/text/numbers)
         |
         v
   INPUT LAYER  (raw features)
         |
         | (weights multiply, add, activate) x many times
         v
  HIDDEN LAYERS  (patterns seekhta hai)
         |
         v
  OUTPUT LAYER  (final prediction)
         |
         v
   LOSS FUNCTION  (kitna galat tha measure karo)
         |
         v (backward pass)
  BACKPROPAGATION  (gradient calculate karo)
         |
         v
  GRADIENT DESCENT  (weights update karo: w = w - eta*grad)
         |
         v
  Baar baar repeat karo jab tak loss minimize na ho jaye
         |
         v
   TRAINED MODEL!  (deploy karo, use karo)
```

Ab aage ke chapters mein ye hi concepts deep mein padhenge. Chapter 0 ne foundation de di hai. Chapter 1 se specifically exam-relevant calculations shuru hoti hain.

CHAPTER 1: CNN Calculations — Sabse Important Topic

Kyun important hai? Teen teeno papers mein 10-16 marks sirf CNN calculations se aate hain. Ye formulas rote lo — free marks hain.

Formula 1: Convolution ke baad Output Size

Ye SABSE PEHLI aur SABSE ZYADA use hone wali formula hai. Har CNN question yahan se shuru hota hai.

```
W_out = (W_in - F + 2P) / S  +  1
H_out = (H_in - F + 2P) / S  +  1
D_out = K   (number of filters / kernels)

Jahan:
  W_in, H_in = Input image ka width, height
  F = Filter/kernel ka size (e.g., 3 means 3x3)
  P = Padding (kitna zero border add kiya)
  S = Stride (filter kitne steps move karta hai)
  K = Kitne filters hain
```

Simple trick yaad karne ka: Agar P=0 aur S=1 (jo mostly exam mein hota hai), to W_out = W_in - F + 1

Worked Example (Dec 2024 Paper se exact)

Example: Input 128x128x3, ConvLayer1: F=9, K=32, S=1, P=0
Step 1: W_out = (128 - 9 + 0)/1 + 1 = 120
Step 2: H_out = (128 - 9 + 0)/1 + 1 = 120
Step 3: D_out = K = 32
Step 4: Feature Map Volume = 120 x 120 x 32 = 460,800 ✓ (Yahi paper ka answer tha!)

Formula 2: Conv Layer ke Parameters

Ye count karta hai ki ek Conv layer mein kitne LEARNABLE WEIGHTS hain.

```
Parameters = F x F x D_in x K

Note: Is course mein BIAS IGNORE KARO (exam always bolta hai 'ignore
bias')
D_in = Input ki depth (channels) — pichli layer ka D_out
```

Example: ConvLayer1 — F=9, D_in=3 (RGB), K=32

Parameters = 9 x 9 x 3 x 32

= 81 x 96

= 7,776 ✓ (Exact answer tha paper mein!)

⚠ **EXAM TRAP:** CNN ke parameters DEPEND NAHI KARTE input image ke width/height par! Sirf filter size, input depth aur number of filters se calculate hote hain. Agar image double ho jaye, parameters same rahenge. (Aug 2025 Paper Q8)

Formula 3: Pooling Layer

```
Output Size: W_out = (W_in - F) / S  +  1
Depth: Same rehti hai (D_out = D_in)
PARAMETERS = 0  (Pooling mein koi learnable weight nahi hota!)
```

Example: PoolLayer1 — F=2, S=2, P=0, Input=120x120x32

W_out = (120 - 2)/2 + 1 = 60

D_out = 32 (same)

Feature Map Volume = 60 x 60 x 32 = 115,200 ✓

Parameters = 0 ✓

Formula 4: Fully Connected (FC) Layer ke Parameters

```
Parameters = Input_neurons x Output_neurons
Input_neurons = W x H x D  (last feature map ko flatten karo)
```

Example: PoolLayer2 ke baad output 28x28x64 hai, FC=3 neurons

Input = 28 x 28 x 64 = 50,176

FC Parameters = 50,176 x 3 = 150,528 ✓

FULL STEP-BY-STEP CNN CHAIN EXAMPLE

Ye exact same question Dec 2024 paper mein Q17 se Q24 tak tha. Poori chain solve karo:

Input: 128x128x3 (RGB image)

Layer	Formula / Calculation	Answer

ConvLayer1 (F=9, K=32, S=1, P=0)	W: (128-9)/1+1=120, D=32	120x120x32
Feature Map Vol	120 x 120 x 32	460,800
Conv1 Parameters	9 x 9 x 3 x 32	7,776
PoolLayer1 (F=2, S=2)	W: (120-2)/2+1=60, D=32	60x60x32
Pool1 Feature Map Vol	60 x 60 x 32	115,200
Pool1 Parameters	Pooling = 0	0
ConvLayer2 (F=5, K=64, S=1, P=0)	W: (60-5)/1+1=56, D=64	56x56x64
Conv2 Feature Map Vol	56 x 56 x 64	200,704
Conv2 Parameters	5 x 5 x 32 x 64 (D_in=32!)	51,200
PoolLayer2 (F=2, S=2)	W: (56-2)/2+1=28, D=64	28x28x64
Pool2 Feature Map Vol	28 x 28 x 64	50,176
FC Layer (3 neurons)	50,176 x 3	150,528

⚠ **EXAM TRAP:** Conv2 ka D_in = 32 hai (PoolLayer1 se aaya), 3 nahi! Bahut common mistake hai. Har layer ke baad depth update karo.

CONV vs FC — Parameter Comparison

Exam ek common MCQ puchta hai: 'Which has fewer parameters?'

Example (Dec 2024 Paper Q14-Q15): Input 10x10x3

FC Layer (2 neurons):

Parameters = (10 x 10 x 3) x 2 = 300 x 2 = 600

CONV Layer (3 filters, F=2x2, S=2, P=0):

Parameters = 2 x 2 x 3 x 3 = 36

Difference = 600 - 36 = 564 ✓

Answer: CONV layer has FEWER parameters!

Kyun? CNN weight sharing karta hai — same filter puri image par apply hoti hai. FC mein har neuron ka alag weight hota hai.

CNN Parameters — Kya depend karta hai, kya nahi?

Statement	True ya False?
Parameters depend karte hain filter size (F) par	TRUE ✓
Parameters depend karte hain input depth (D_in) par	TRUE ✓
Parameters depend karte hain number of filters (K) par	TRUE ✓
Parameters depend karte hain padding par	FALSE ✗
Parameters depend karte hain stride par	FALSE ✗
Parameters depend karte hain input image size par	FALSE ✗

Padding ke saath output size

Agar padding add karo, to output size badi ho jati hai:

```
With Padding: W_out = (W_in - F + 2P) / S + 1


Same size output chahiye (W_out = W_in)?
  P = (F-1)/2  set karo (for S=1)
  Example: F=3 => P=1, F=5 => P=2
```

Practical Example: Input 28x28x256, F=1, K=128, S=1, P=0

W_out = (28 - 1 + 0)/1 + 1 = 28

D_out = 128

Volume = 28 x 28 x 128 = 100,352 ✓ (Mar 2025 Paper Q16)

GoogLeNet Inception Module — Special Case

Ye ek special CNN architecture hai jahan ek hi input par MULTIPLE filter sizes parallel mein chalti hain, phir outputs concatenate hoti hain.

```
Final depth after concatenation = sum of all K values from each branch


Example: 4 branches se K = 128 + 192 + 96 + 64 = 480
Width/Height wahi rehti hai (padding se maintain ki jaati hai)
```

Mar 2025 Paper Q24 — Final concatenated output?

Branch 1 (F=1): K=128, output W=28

Branch 2 (F=3, P=1): K=192, output W=28

Branch 3 (F=5, P=2): K=96, output W=28

Branch 4 (MaxPool + F=1): K=64, output W=28

Final = 28 x 28 x (128+192+96+64) = 28 x 28 x 480 ✓

Quick Practice Questions

Q: Input 64x64x8, Conv: F=5, K=16, S=1, P=2. Output shape kya hoga?

A: W_out = (64-5+4)/1+1 = 64. Output = 64x64x16. Parameters = 5x5x8x16 = 3200

Q: Ek image 32x32x3 hai. MaxPool F=2, S=2 apply karo. Output volume kya?

A: W_out = (32-2)/2+1 = 16. Volume = 16x16x3. Parameters = 0

Q: Conv layer: F=3, K=64, S=1, P=1, Input 28x28x32. Output size aur parameters?

A: W_out = (28-3+2)/1+1 = 28. Output = 28x28x64. Params = 3x3x32x64 = 18,432

CHAPTER 2: Optimizers — Numerical Questions

Kyun important hai? Teeno papers mein AdaGrad ke step-by-step numerical questions the. Ye formulas yaad karo aur 5+ marks pakke.

Pehle Samjho: Optimization Problem Kya Hai?

Humara goal hai neural network ka loss function minimize karna. Loss function ek pahaad ki tarah hai — hum neeche jaana chahte hain.

Problem	Solution
Flat surface par GD bahut slow chal ta hai	Momentum use karo — speed badhao
Steep slopes par learning rate bahut bada ho jata hai	Adaptive LR use karo — gradient ke hisab se LR adjust karo
AdaGrad mein LR zero ho jata hai eventually	RMSProp — exponential average use karo
RMSProp mein direction fluctuate karta hai	Adam — momentum bhi add karo

Vanilla Gradient Descent (Base Case)

```
w_new = w_old - eta * gradient(w_old)


jahan eta = learning rate (step size)

gradient = dL/dw  (loss ka derivative)
```

Simple example: $L(w) = w^2$, $w_0=2$, eta=0.1
gradient = 2w = 4 | $w_1 = 2 - 0.1\times4 = 1.6$

ADAGRAD — SABSE ZYADA TESTED

Key idea: Har parameter ke liye alag learning rate — jo gradient zyada tha uska LR chhota kar do.

```
STEP 1: v_t = v_{t-1} + (gradient_t)²    <-- squared gradient accumulate
karo

STEP 2: w_{t+1} = w_t - (eta / sqrt(v_t + epsilon)) * gradient_t


Initial conditions: v_{-1} = 0, epsilon = 0 (exam mein usually)
```

Poora Worked Example (Dec 2024 Paper Q2-Q4 — Exact)

Given: $L(w) = w^2$, eta=1, w_0=2, epsilon=0

Gradient formula: $dL/dw = 2w$

t=0 (Pehla iteration):

gradient at $w_0 = 2\times2 = 4$

$v_0 = v_{-1} + (4)^2 = 0 + 16 = 16$

$w_1 = w_0 - (1/\text{sqrt}(16)) \times 4 = 2 - (1/4)\times4 = 2 - 1 = 1$ ✓ (Answer: w_1=1)

t=1 (Doosra iteration):

gradient at $w_1 = 2\times1 = 2$

$v_1 = v_0 + (2)^2 = 16 + 4 = 20$ ✓ (Answer: v_1=20)

$w_2 = w_1 - (1/\text{sqrt}(20)) \times 2 = 1 - 2/4.472 = 1 - 0.447 = 0.553$ ✓ (Range: 0.549-0.556)

⚠ **EXAM TRAP:** v_1 matlab w_1 par calculate kiya gaya v — w_0 par nahi! Subscript confusing lagta hai but v subscript w subscript se match karta hai.

Another Example (Mar 2025 Paper Q12-Q15) — $L(w) = w^3 + 2w$

Given: $L(w) = w^3+2w$, eta=0.5, w_0=2, epsilon=0

Gradient formula: $dL/dw = 3w^2 + 2$

t=0:

gradient at $w_0 = 3\times(4) + 2 = 14$

$v_0 = 0 + 14^2 = 196$ ... wait, paper says 272

Oh! Sahi hai: $3\times(2)^2 + 2 = 3\times4+2 = 14$. $14^2 = 196$.

Paper answer ~272 means gradient = sqrt(272) ≈ 16.5.

Actually: $dL/dw = 3w^2 + 2$. At w=2: 3(4)+2 = 14. $v_0 = 196$.

$w_1 = 2 - 0.5/\text{sqrt}(196) \times 14 = 2 - 0.5/14 \times 14 = 2 - 0.5 = 1.5$ ✓ (Range 1.4-1.6)

t=1: gradient at w_1=1.5 → 3(2.25)+2 = 8.75

$v_1 = 196 + 8.75^2 = 196 + 76.56 = 272.56$ ✓ (Range 272.46-272.66)

RMSPROP

Key idea: AdaGrad zyada aggressive hai — LR eventually zero ho jata hai. RMSProp exponential moving average use karta hai.

```
v_t = beta × v_{t-1} + (1-beta) × (gradient_t)²
w_{t+1} = w_t - (eta / sqrt(v_t + epsilon)) × gradient_t

Difference from AdaGrad: beta term se puraane gradients ko discount karo
Typical beta = 0.9
```

WEIGHTED MOVING AVERAGE (WMA) — Mar 2025 Q8

```
v_t = beta × v_{t-1} + (1-beta) × gradient_t
v_{-1} = 0
```

Example: beta=0.9, gradients: w_0=5, w_1=3, w_2=4. Find v_2.

v_0 = 0.9×0 + 0.1×5 = 0.5

v_1 = 0.9×0.5 + 0.1×3 = 0.45 + 0.3 = 0.75

v_2 = 0.9×0.75 + 0.1×4 = 0.675 + 0.4 = 1.075 ✓ (Range 1.065-1.085)

ADAM — Adaptive Moments

Key idea: RMSProp + Momentum ka combination. Iske alawa bias correction bhi karta hai.

```
m_t = beta1×m_{t-1} + (1-beta1)×gradient    <-- 1st moment (momentum)
v_t = beta2×v_{t-1} + (1-beta2)×(gradient)² <-- 2nd moment (RMSProp)

Bias Correction (important!):
  m_hat = m_t / (1 - beta1^t)
  v_hat = v_t / (1 - beta2^t)

Update: w_{t+1} = w_t - (eta / sqrt(v_hat + epsilon)) × m_hat
```

Bias correction kyun? Starting mein m aur v 0 se initialize hote hain, to pehle few iterations mein ye bahut underestimate karte hain. Divide karne se correct ho jata hai. (Mar 2025 Paper Q3)

⚠ **EXAM TRAP:** Adam = RMSProp + Momentum. 'Which optimizer combines RMSProp and momentum?' ka answer HAMESHA Adam hai.

ADAMAX

Adam ka ek variation — L2 norm ki jagah L-infinity norm use karta hai.

```
v_t = max(beta2 × v_{t-1}, |gradient_t|)  <-- max operation
w_{t+1} = w_t - (eta / v_t) × m_hat
Note: AdaMax mein bias correction v_t par nahi hoti
```

NADAM

Adam + NAG (Nesterov) ka combination.
NAG 'look ahead' karta hai — pehle momentum wala step lo, phir gradient compute karo.

Momentum Based GD

```
u_t = beta × u_{t-1} + gradient_t
w_{t+1} = w_t - eta × u_t

Intuition: Ball jo slope pe gir rahi hai — momentum build up karta hai
beta typically 0.9 ke aas paas hota hai
```

NAG — Nesterov Accelerated Gradient

```
u_t = beta × u_{t-1} + gradient(w_t - beta × u_{t-1})
w_{t+1} = w_t - eta × u_t

Difference from Momentum: Gradient compute karne se PEHLE ek tentative
step lo
Isse oscillations kam hote hain
```

Optimizer Comparison Table — MCQ/MSQ ke liye

Optimizer	Adaptive LR?	Momentum?	Key Problem Solved
Vanilla GD	No	No	Baseline
Momentum	No	Yes	Slow flat surfaces
NAG	No	Yes (look-ahead)	Oscillations in Momentum
AdaGrad	Yes	No	Sparse features, freq params
RMSProp	Yes	No	AdaGrad dying LR problem
Adam	Yes	Yes	Best of RMSProp + Momentum
AdaMax	Yes (L-inf)	Yes	Adam with max norm
NAdam	Yes	Yes (NAG)	Adam + look-ahead

EXAM MSQ TRAP

⚠ **EXAM TRAP:** 'Which optimizers use ADAPTIVE LEARNING RATE?' = AdaGrad, RMSProp, Adam, AdaDelta — YES. Momentum aur NAG = NO, inme LR adapt nahi hoti. (Aug 2025 Paper Q3)

Learning Rate Scheduling

Ye optimizer nahi hain — ye sirf eta (learning rate) ko time ke saath change karte hain:

Method	Kaise kaam karta hai
Step Decay	Har kuch epochs baad LR half kar do
Exponential Decay	eta = eta_0 × e^(-kt)
1/t Decay	eta = eta_0 / (1 + kt)
Cosine Annealing	LR cosine wave ki tarah oscillate karta hai
Cyclical LR	LR up-down karta rehta hai saddle points avoid karne ke liye

Cyclical LR kyun? Saddle points pe normal LR zero ho jaata hai aur model stuck ho jaata hai. LR increase karne se model bahar nikal sakta hai.

Practice Questions

Q: L(w)=w^2, eta=0.5, w_0=3, epsilon=0. AdaGrad se w_1 aur v_0 nikalo.

A: gradient at w_0=3: dL/dw=2w=6. v_0=0+36=36.
w_1=3-(0.5/sqrt(36))×6=3-(0.5/6)×6=3-0.5=2.5

Q: Kaunsa optimizer RMSProp aur Momentum dono combine karta hai?

A: ADAM

Q: Adam mein bias correction kyun zaroori hai?

A: Starting mein m_1=(1-beta1)×gradient bahut small hota hai — underestimate karta hai true mean. Division by (1-beta1^t) se correct hota hai. (Mar 2025 Q3 exact answer)

CHAPTER 3: Backpropagation aur Loss Functions

Feed Forward Network — Structure Samjho

Pehle forward pass samjho, tabhi backward pass samjhega.

```
Layer i ke liye:
  Pre-activation: a_i = b_i + W_i × h_{i-1}
  Activation:     h_i = g(a_i)   [g = ReLU, sigmoid, tanh]

Output Layer:
  Classification: y_hat = softmax(a_L)
  Regression:     y_hat = a_L (linear)

h_0 = x (input)    h_L = y_hat (output)
```

Loss Functions

Task	Output + Loss
Regression	Output: Linear | Loss: MSE = (1/N) Σ (y_hat - y)²
Binary Classification	Output: Sigmoid | Loss: Binary Cross-Entropy
Multi-class Classification	Output: Softmax | Loss: Cross-Entropy = -Σ y_i × log(y_hat_i)

```
Softmax: y_hat_j = exp(a_j) / sum_k(exp(a_k))

Cross-Entropy (single sample, true class = l):
  L = -log(y_hat_l)

MSE: L = (1/N) × sum_i (y_hat_i - y_i)²
```

Backpropagation — Chain Rule

Key idea: Loss ko har weight ke respect mein differentiate karo, then gradient descent use karo.

```
Chain Rule: dL/dW = dL/da × da/dW = dL/da × h_{prev}

Output layer gradient (softmax + cross-entropy):
```

```
  dL/da_L = -(e_y - y_hat)   [simple formula!]
  e_y = one-hot vector (true class ki position 1)

Hidden layer gradient:
  dL/dh_i = W_{i+1}^T × dL/da_{i+1}
  dL/da_i = dL/dh_i × g'(a_i)
```

Activation Derivatives (Zaroor yaad karo!):

Sigmoid: sigma'(x) = sigma(x) × (1 - sigma(x))
= 0 when x is very large or very small (saturated)

Tanh: tanh'(x) = 1 - tanh²(x)

ReLU: f'(x) = 1 if x > 0
0 if x <= 0 <-- DEAD NEURON!

Leaky ReLU: f'(x) = 1 if x > 0
0.01 if x < 0

Dead Neuron Problem — Exam Question (Dec 2024 Q13)

Concept: Agar ReLU neuron ka pre-activation ≤ 0 hai, to uska gradient = 0. Iska matlab wo neuron 'dead' hai — na forward pass mein contribute karta hai, na backward pass mein.

Dec 2024 Q13 — Which weight matrices get non-zero gradients?

Network: W1→h1→W2→h2→W3→h3(sigmoid)→W4→output

h2 = [3, 2, 0, 1, 6] (0 wala neuron DEAD hai!)

ReLU definition used: f(x) = x if x>=1, else 0
So neurons with value <1 are dead...

Key rule: Agar kisi neuron ka output 0 hai, to:
- Us neuron ke through gradient flow NAHI karega
- W3 ko gradient milega (h3 sigmoid hai, sigmoid never fully dead)
- W4 ko gradient milega
- W1, W2 ko partial gradient milega (non-zero neurons se)

Answer: W3 aur W4 definitely non-zero gradients lete hain

Backprop Through CNN

Jab CNN mein backprop hota hai, convolution operation bhi differentiate hoti hai.

```
Convolution layer ka gradient (kernel W ke respect mein):
  dL/dW_{ij} = sum over positions where W_{ij} was used
             = sum_{m,n} (dL/d_output[m,n]) × input[m+i, n+j]

MaxPool gradient:
  Gradient sirf max value ki position par jaata hai
  Baaki positions ka gradient = 0

AvgPool gradient:
  Gradient equally divide hota hai sab positions mein
```

Mar 2025 Q11 — $\partial L/\partial K_{00}$ kaise nikalo:

Step 1: Convolution apply karo X aur K se — output A nikalo

Step 2: A ko sigmoid se pass karo — y_hat nikalo

Step 3: Given $\partial L/\partial y_hat = -1$

Step 4: Chain rule: $\partial L/\partial K_{00} = \partial L/\partial y_hat \times \partial y_hat/\partial A \times \partial A/\partial K_{00}$

$= -1 \times sigma(A)(1-sigma(A)) \times$ (input patch jo K_{00} ne touch kiya)

Answer: ~-0.126 (Range -0.136 to -0.116)

Autoencoder aur Tied Weights

Concept: Autoencoder mein input aur output same size ka hota hai. Hidden layer compressed representation seek ta hai.

Aug 2025 Q4 — Tied vs Untied weights:

Input: n neurons, Hidden: 0.3n neurons, Output: n neurons

Untied ($W_1 \neq W_2$):

W_1: $n \times 0.3n = 0.3n^2$ parameters

W_2: $0.3n \times n = 0.3n^2$ parameters

Total $n_1 = 0.6n^2$

Tied ($W_2 = W_1^T$):

Sirf W_1 store karo: $0.3n^2$ parameters

Total $n_2 = 0.3n^2$

Ratio $n_1/n_2 = 0.6n^2 / 0.3n^2 = 2$ ✓

Practice Questions

Q: Sigmoid neuron fully saturate ho jaata hai (output ≈ 0 ya 1). Gradient kya hoga?

A: Gradient ≈ 0. Sigmoid derivative = sigma(1-sigma). Jab sigma≈0 ya sigma≈1, product ≈ 0. Ye VANISHING GRADIENT problem hai.

Q: MSE loss kab use karte hain? Cross-entropy kab?

A: MSE: Regression mein (continuous output). Cross-entropy: Classification mein (probability output).

CHAPTER 4: Activation Functions, Bias-Variance, Regularization

Activation Functions — Complete Table

Ye table har exam mein kisi na kisi form mein aata hai. Roto mat, samjho.

Function	Formula	Range	Derivative	Problem / Use Case
Sigmoid	$1/(1+e^{-x})$	(0, 1)	$\sigma(x)(1-\sigma(x))$	Vanishing gradient, not zero-centered
Tanh	tanh(x)	(−1, 1)	$1-\tanh^2(x)$	Vanishing gradient (better than sigmoid)
ReLU	max(0, x)	[0, ∞)	1 (x>0), 0 (x≤0)	Dying neurons; x<0 => dead
Leaky ReLU	max(0.01x, x)	(−∞, ∞)	1 (x>0), 0.01 (x<0)	Fixed dead neuron problem
Param. ReLU	max(αx, x)	(−∞, ∞)	1 (x>0), α (x<0)	α learned during training
ELU	x (x>0), $a(e^x-1)$ (x≤0)	(−a, ∞)	Complex	Costly computation
GELU	x×Φ(x)	~(−0.17, ∞)	Complex	Used in BERT/GPT
Swish	x×σ(βx)	~(−0.28, ∞)	Complex	β=1.702 gives GELU

⚠ **EXAM TRAP:** Exam Q: 'Which activation functions are prone to VANISHING GRADIENT?' — Sigmoid ✓, Tanh ✓, ReLU ✓ (ReLU mein dying neuron = permanent zero gradient, which is even worse). Leaky ReLU ✗, GELU ✗, MaxOut ✗

⚠ **EXAM TRAP:** Leaky ReLU derivative = 0.01 for negative inputs (NOT 0.1). 0.1 sirf slope hai jab alpha default 0.1 use ho. Exam mein typically alpha=0.01.

Maxout

Ye ReLU aur Leaky ReLU ka generalization hai. k neurons ka maximum propagate karta hai. Dropout ke saath well kaam karta hai.

BIAS aur VARIANCE — Loss Curves Padhna

Ye concept paper ke MCQ mein visuals ke saath aata hai. Loss curves dekh ke identify karo:

Pattern (Loss Curves mein)	Matlab kya hai?
Train loss LOW, Val loss BADA gap (val>>train)	LOW BIAS, HIGH VARIANCE = Overfitting
Train loss HIGH, Val loss bhi HIGH, dono milte hain	HIGH BIAS, LOW VARIANCE = Underfitting

Train loss aur Val loss dono LOW, milte hain	LOW BIAS, LOW VARIANCE = Good model!
Loss oscillate karta hai, converge nahi karta	HIGH LEARNING RATE problem
Training start mein val < train, phir reverse	Also HIGH VARIANCE / overfitting

Aug 2025 Paper Q2 — M1, M2, M3, M4 matching:

M1: Train falls, Val ≈ Train (both low by end) → (C) Low bias, Low variance

M2: Both rise and diverge sharply → (A) Learning rate too high

M3: Train falls fast, Val stays high → (B) Low bias, High variance

M4: Both stay high, don't converge → (D) High bias, Low variance

Bias-Variance Trade-off — Mathematical Formula

```
E[(y - y_hat)²] = Bias² + Variance + sigma²(irreducible noise)

Bias = E[y_hat] - y_true  (average prediction kitna door hai truth se)
Variance = E[(y_hat - E[y_hat])²]  (predictions kitni spread out hain)

Simple model: High Bias, Low Variance
Complex model: Low Bias, High Variance
```

Mar 2025 Q7 — Variance Calculate karna:

3 models, test = [1,3,6,8]

Model A: [1,4,7,8], B: [3,5,6,9], C: [2,3,5,7]

For each test point, find average prediction:

Point 1: avg = (1+3+2)/3 = 2, variance = $((1-2)^2+(3-2)^2+(2-2)^2)/3 = 2/3$

Point 2: avg = (4+5+3)/3 = 4, variance = $((4-4)^2+(5-4)^2+(3-4)^2)/3 = 2/3$

Point 3: avg = (7+6+5)/3 = 6, variance = $((7-6)^2+(6-6)^2+(5-6)^2)/3 = 2/3$

Point 4: avg = (8+9+7)/3 = 8, variance = $((8-8)^2+(9-8)^2+(7-8)^2)/3 = 2/3$

Average variance over all points = 2/3 ≈ 0.667 ✓ (Range 0.62-0.72)

REGULARIZATION — Overfitting Rokna

L2 Regularization (Weight Decay)

```
Regularized Loss: L_bar(w) = L(w) + (alpha/2) × ||w||²

Gradient: dL_bar/dw = dL/dw + alpha × w

Update: w_new = w_old - eta × (dL/dw + alpha × w)
      = (1 - eta×alpha) × w_old - eta × dL/dw
```

Effect: Weights 0 ki taraf push hote hain. Important weights kam affect hote hain, unimportant weights zyada. L2 weights ko exactly 0 nahi banata (that's L1).

L2 Regularization — Mathematical Property (Aug 2025 Q9-Q11)

Given: $L_D(w) = w_1^2+w_2^2-2w_1-2w_2+2$, $L_R = (lambda/2)||w||^2$

Find w*_D (unregularized minimum):

$dL_D/dw_1 = 2w_1 - 2 = 0 \rightarrow w_1 = 1$

$dL_D/dw_2 = 2w_2 - 2 = 0 \rightarrow w_2 = 1$

w*_D = (1, 1) ✓

Find lambda if w* = (0.5, 0.5):

Total loss L = L_D + L_R

$dL/dw_1 = 2w_1-2 + lambda \times w_1 = 0$

At $w_1=0.5$: 2(0.5)-2 + lambda(0.5) = 0

1-2 + 0.5×lambda = 0 → lambda = 2 ✓

Is ||w*|| > ||w*_D|| possible? NO! L2 hamesha weights shrink karta hai.

Answer: No such lambda exists ✓

Dropout

Idea: Training mein randomly kuch neurons drop kar do (temporarily hata do). Har batch mein alag 'thinned' network milti hai. Ye bagging jaisa hai without training multiple models.

Dropout ka aspect	Detail
Probability	Hidden units: p=0.5, Input units: p=0.8
During Training	Random masking — kuch neurons zero kar do
During Testing	Sab neurons active, outputs scale karo
Effect	Co-adaptation rokta hai — neurons independent seekhte hain
Equivalence	Masking noise inject karna = ensemble methods

Other Regularization Methods

Method	Kaise kaam karta hai
Data Augmentation	Images rotate, flip, crop karo — zyada data lagti hai. Speech mein bhi kaam karta hai.
Early Stopping	Validation loss badhne lage to rok do. Patience parameter set karo.
L1 Regularization	Penalty = |w|. Weights exact 0 ho sakte hain — sparse model.
Noise Injection (Input)	Input mein Gaussian noise add karo = L2 regularization jaisa effect
Noise Injection (Output)	Soft targets use karo instead of hard 0/1 labels
Parameter Sharing	CNNs mein same filter = weights share karna = regularization
Batch Normalization	Layer ke activations normalize karo — overfitting kam hota hai

Exam MSQ: 'Which are regularization techniques?' (Dec 2024 Q6)

Dataset augmentation ✓

L2 regularization ✓

Adding noise to inputs/outputs ✓

Activation functions ✗ (ye regularization nahi hai)

Optimization techniques ✗ (ye regularization nahi hai)

Cross-Entropy Loss with Noisy Labels (Aug 2025 Q5)

5-class problem, true label y=(1,0,0,0,0), noisy label y_noise=(0.9,0.025,0.025,0.025,0.025)

Network prediction: y_hat = (0.6, 0.1, 0.1, 0.1, 0.1)

Cross-entropy loss = -sum_i y_noise_i × log(y_hat_i)

= -[0.9×log(0.6) + 0.025×log(0.1) + 0.025×log(0.1) + 0.025×log(0.1) + 0.025×log(0.1)]

= -[0.9×(-0.5108) + 4×0.025×(-2.3026)]

= -[-0.4597 - 0.2303]

= -[-0.6900] = 0.69 ✓ (Range 0.64-0.74)

Practice Questions

Q: Simple model ka bias HIGH hoga ya LOW?

A: HIGH bias. Simple model data ke complex patterns capture nahi kar sakta, is liye predictions consistently galat hoti hain.

Q: L2 regularization kya karta hai weights ke saath?

A: Weights ko 0 ki taraf push karta hai (shrink karta hai). Gradient mein +alpha×w add hota hai. Weights exactly 0 nahi hote (that's L1).

Q: Dropout testing time mein differently kyun kaam karta hai training se?

A: Training mein neurons randomly drop hote hain. Testing mein sab active hote hain but outputs scale ho jaate hain taaki expected value match kare.

CHAPTER 5: Convolution Operation aur CNN Architectures

Convolution Operation — By Hand

Recent papers mein actual matrix convolution hand-calculate karna aata hai.

```
Correlation (libraries ye use karti hain, convolution nahi):
output[i, j] = sum_{a,b} Input[i+a, j+b] × Kernel[a, b]

True Convolution (kernel flip karo):
output[i, j] = sum_{a,b} Input[i-a, j-b] × Kernel[a, b]

Practical note: Libraries correlation use karti hain, exam bhi mostly
correlation hai
Dono same result dete hain agar kernel symmetric ho
```

Step-by-Step Convolution (Mar 2025 Q9)

X=4x4 matrix, K=3x3 kernel, S=1, P=0 → Output = 2x2

Output size: (4-3)/1 + 1 = 2. So 2x2 output.

X = [1,0,-2,1; 1,2,0,1; -1,-1,1,0; 0,1,0,1]

K = [1,0,1; 0,0,1; 1,0,1]

Output[0,0] = element-wise X[0:3, 0:3] dot K

= 1×1 + 0×0 + (-2)×1 + 1×0 + 2×0 + 0×1 + (-1)×1 + (-1)×0 + 1×1

= 1 + 0 - 2 + 0 + 0 + 0 - 1 + 0 + 1 = -1

Output[0,1] = X[0:3, 1:4] dot K = similar calculation = 1

Output[1,0] = X[1:4, 0:3] dot K = 2

Output[1,1] = X[1:4, 1:4] dot K = 5

Result A = [-1, 1; 2, 5] ✓ (Mar 2025 Paper answer)

Average Pooling Example (Mar 2025 Q10)

Apply 2x2 average pooling on A = [-1,1; 2,5]

Average pooling 2x2 on A = single value (kyunki A hi 2x2 hai)

= (-1 + 1 + 2 + 5) / 4 = 7/4 = 1.75 ✓

CNN ke Important Properties

Property	Explanation
Weight Sharing	Same filter puri image par slide karta hai. Isliye parameters kam hain.
Local Connectivity	Har neuron sirf apne receptive field se connected hai, puri image se nahi.
Translation Invariance	Same features anywhere bhi detect ho sakti hain.
Depth = Number of Filters	Har filter ek feature detect karta hai — edges, curves, textures, etc.

CNN Architectures — MCQ ke liye

Architecture	Key Identifying Feature
AlexNet	First successful deep CNN. ReLU use kiya. ~27M parameters. 5 Conv + 3 FC layers.
ZFNet	AlexNet jaisi but 3 middle conv layers ke filters 512, 1024, 512.
VGGNet	HAMESHA 3x3 filters. Depth zyada, filter size chhota.
GoogLeNet/Inception	PARALLEL branches — multiple filter sizes (1x1, 3x3, 5x5) ek saath. Output concatenate karo.
ResNet	SKIP CONNECTIONS — input of layer + output of layer. Element-wise addition.

⚠ **EXAM TRAP:** ResNet skip connections = ELEMENT-WISE ADDITION (concatenation nahi, multiplication nahi). (Mar 2025 Paper Q4 exact)

ResNet — Skip Connection Samjho

```
Normal layer:  output = F(input, W)  [just transformation]


ResNet layer:  output = F(input, W) + input  [add skip connection]


Agar F(input, W) ≈ 0 bhi ho jaaye, output = input (identity mapping)
Isse gradient easily flow karta hai deep networks mein bhi
Yahi ResNet ka genius hai — very deep networks train ho paati hain
```

GoogLeNet Inception Module — Poora Samjho

Ek inception module mein SAME INPUT par 4 parallel operations hoti hain:

Branch	Operation
Branch 1	1x1 convolution → dimensionality reduction
Branch 2	1x1 conv (reduce) → 3x3 conv
Branch 3	1x1 conv (reduce) → 5x5 conv
Branch 4	3x3 MaxPool → 1x1 conv

Sab branches ke outputs ko DEPTH dimension mein concatenate karo.
Final depth = sum of all K values from each branch.

Pooling Types

Type	Kaam aur Properties
Max Pooling	Region ka maximum value lo. Backprop: gradient sirf max position par jaata hai.
Average Pooling	Region ka average lo. Backprop: gradient equally divide hota hai.
Global Average Pooling	Puri feature map ka average ek value — FC layer replace karta hai.

Practice Questions

Q: Input 5x5, Kernel 3x3, S=1, P=0. Output size kya hoga?

A: (5-3)/1+1 = 3. Output = 3x3

Q: VGGNet ki key feature kya hai?

A: Hamesha 3x3 filters use karta hai. Deep architecture hai.

Q: MaxPool ke through backprop mein kya hota hai?

A: Gradient sirf wo position leta hai jahan maximum value tha. Baaki positions ka gradient = 0.

Q: Skip connections kaise kaam karte hain ResNet mein?

A: F(x) + x. Input directly output se add hota hai (element-wise). Isse gradient easily backpropagate karta hai.

CHAPTER 6: History aur Foundations (Brief)

Note: Ye topics exam mein kam aate hain lekin MCQ mein concepts puch sakte hain.

MP (McCulloch-Pitts) Neuron

```
Inputs: Binary (0 ya 1)
Inhibitory input: Agar koi bhi inhibitory input = 1 → output = 0
Otherwise: y = 1 if sum(x_i) >= theta, else 0

Kya kar sakta hai: AND, OR, NOT, NOR (linearly separable functions)
Kya nahi kar sakta: XOR (not linearly separable)
```

Perceptron

MP Neuron ka improvement: real-valued inputs + learnable weights.

```
y = 1 if sum(w_i × x_i) >= 0, else 0  (bias include karne ke baad)

MP Neuron se difference:
  - Inputs real-valued ho sakte hain (sirf binary nahi)
  - Weights learnable hain (fixed nahi)

Still only linearly separable functions!
```

Boolean Functions — Important Facts:

n inputs se: 2^(2^n) total boolean functions possible

n=2 inputs: 2^4 = 16 functions

In of 16 functions: sirf XOR aur XNOR not linearly separable

Theorem: Any boolean function of n inputs = 1 hidden layer with 2^n perceptrons

Catch: Exponential! 20 inputs = 1 million perceptrons needed

Sigmoid Neuron — Smooth Decision

Perceptron ki harsh step function ki jagah smooth sigmoid use karo:

```
y = 1 / (1 + exp(-(w0 + sum(w_i × x_i))))
```

```
Universal Approximation Theorem:

Single hidden layer sigmoid network = any continuous function approximate
kar sakta hai
```

CHAPTER 7: RAPID REVISION SHEET — Exam se Pehle Padhna

Ye page EXAM MORNING padhna hai. Sab kuch yahan hai.

10 Formulas — ROTO LO

```
1. Conv output size:    W_out = (W_in - F + 2P)/S + 1
2. Conv parameters:     F x F x D_in x K  (no bias)
3. Pooling params:      = 0  (hamesha!)
4. FC parameters:       flatten × output_neurons
5. AdaGrad Step 1:      v_t = v_{t-1} + (gradient)²
6. AdaGrad Step 2:      w_new = w - eta/sqrt(v_t) × gradient
7. RMSProp:             v_t = beta×v_{t-1} + (1-beta)×(gradient)²
8. Adam:                momentum + RMSProp + bias correction
9. Sigmoid deriv:       sigma(x) × (1-sigma(x))
10. L2 loss:            L(w) + (alpha/2)||w||²  → gradient adds alpha×w
```

Quick MCQ Bank — Ye questions CONFIRM aate hain

Question	Answer
RMSProp + Momentum = ?	ADAM
Adaptive LR wale: (MSQ)	AdaGrad, RMSProp, Adam (NOT Momentum, NOT NAG)
Vanishing gradient wale activations:	Sigmoid, Tanh, ReLU (dying)
Leaky ReLU derivative (negative input):	0.01
L2 regularization add karta hai:	Sum of SQUARED weights penalty
ResNet skip connection =	Element-wise ADDITION (not concat)
Pooling parameters =	ZERO
CNN params depend on image size?	NO — same rehte hain
Simple model: bias/variance?	High bias, Low variance
Complex model: bias/variance?	Low bias, High variance
Adam mein bias correction kyun?	Early iterations mein underestimate correct karne
XOR linearly separable hai?	NO — MLP chahiye
Soft target kya hota hai?	Continuous probability, not hard 0/1
n inputs se bool functions?	2^(2^n)

LAST MINUTE TRICKS

Ye galtiyan mat karna exam mein:

1. Conv2 ka D_in = Pool1 ka D_out (pichli layer ki depth use karo, input image ka nahi!)
2. v subscript aur w subscript match karte hain — v_1 matlab w_1 par compute kiya
3. Pooling ke koi parameters nahi hote — ZERO!
4. 'Ignore bias' exam mein standard hai — bias add mat karo parameters mein
5. Feature map volume = W × H × D (multiply karo, add nahi)
6. GoogLeNet final depth = SUM of all branch K values
7. L2 weights shrink karta hai, exactly zero nahi karta (that's L1!)
8. Dropout testing mein outputs scale hote hain
9. ReLU derivative 0 (<= 0) aur 1 (> 0) — binary
10. Adam = combine of RMSProp + Momentum — ye trick question hai!

Exam Strategy

30 marks guaranteed karne ka plan:

CNN numerical questions (10-14 marks): Formulas memorize karo, chain follow karo
→ Sirf ye ek skill se 35% marks aa jaate hain

Optimizer numerical (4-6 marks): AdaGrad steps practice karo
→ Yahan se 15% marks aate hain

MCQ/MSQ (16-20 marks): MCQ bank yaad karo
→ Correct options already notes mein hain

Total: ~40/50 possible agar ye teeno achieve kar lo

Past Paper Practice Solutions

Dec 2024 Paper — Key Solutions:

Q no.	Answer aur Logic
Q2 (w_1)	L=w^2, eta=1, w_0=2. grad=4, v_0=16, w_1=2-1=1
Q3 (v_1)	At w_1=1: grad=2, v_1=16+4=20
Q4 (w_2)	At w_1=1: w_2=1-2/sqrt(20)=0.553
Q5 (Adaptive)	B=RMSProp, D=Adam, E=AdaDelta (all adapt LR)

Q6 (Regularization)	A=Augmentation, C=L2, D=Noise injection
Q7 (Vanishing)	A=Sigmoid, B=Tanh, C=ReLU (dying = zero grad)
Q8 (Adam)	B=Adam (combines RMSProp + Momentum)
Q10 (Leaky ReLU)	C = 0.01 negative, 1 positive
Q17 (Conv1 vol)	120×120×32=460,800
Q18 (Conv1 params)	9×9×3×32=7,776
Q19 (Pool1 vol)	60×60×32=115,200
Q20 (Pool1 params)	0 (pooling!)
Q21 (Conv2 vol)	56×56×64=200,704
Q22 (Conv2 params)	5×5×32×64=51,200
Q23 (Pool2 vol)	28×28×64=50,176
Q24 (FC params)	50,176×3=150,528

Aug 2025 Paper — Key Solutions:

Q no.	Answer aur Logic
Q2 (Loss curves)	M1→C, M2→A, M3→B, M4→D
Q3 (Adaptive)	A=ADAM, B=RMSProp
Q4 (Autoencoder)	$n_1/n_2 = 0.6n^2/0.3n^2 = 2$
Q6 (Conv output)	64×64×16 (S=1, P=2 maintains size)
Q7 (Conv params)	5×5×8×16 = 3200
Q8 (Image size change?)	NO, params same rehte hain
Q9 (w*_D)	(1,1) — minimize L_D
Q10 (lambda)	2 — use gradient = 0 at w*=(0.5,0.5)

Q11 (||w*||>||w*_D||?)	NO — L2 always shrinks, no lambda works

APPENDIX: History Timeline (Sirf Names yaad karo)

Note: History se direct MCQ bahut kam aata hai. Agar time ho tabhi padhna.

Name / Term	Matlab kya hai
McCulloch-Pitts Neuron (1943)	First computational neuron model — binary, threshold
Rosenblatt Perceptron	Learnable weights, real inputs
Book 'Perceptrons' by Minsky	Showed XOR can't be solved → AI Winter
Backpropagation	Gradient efficiently compute karne ka algorithm
Universal Approx. Theorem	Single hidden layer = any function approximate
Hubel & Wiesel experiment	Each neuron has receptive field → inspiration for CNN
AlexNet (2012)	ImageNet jeet, deep learning revival
LSTM (Hochreiter 1997)	Long-range dependencies, vanishing gradient solve
Attention Mechanism	Seq2seq mein relevant words par focus
Transformer (2017)	Attention only — no RNN, parallel processing
GPT / BERT	Transformer based language models
Diffusion Models	GANs se better generation — non-equilibrium thermodynamics
AlphaFold	Protein structure prediction

All the best!

CNN formulas yaad karo, AdaGrad practice karo, MCQ bank dekho.

Paas ho jayega!